Facultad de Ingeniería

Diseño físico de una base de datos

Tema VI

Semestre 2027-1

UNAM
UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Objetivo

El alumno comprenderá y aplicará los elementos necesarios para la implementación física del diseño lógico de la base de datos a través del lenguaje SQL, así como la manipulación y uso de transacciones a través de sentencias.

Introducción

SQL, acrónimo de Structured Query Language, es un lenguaje que permite interactuar con los DBMS para realizar operaciones sobre los datos o sobre la estructura de los datos.

Historia

En los laboratorios de IBM, en los años 70’s, se trabajaba en el desarrollo de un lenguaje que se adaptara a las características del modelo relacional, produciendo un lenguaje llamado sequel.

Historia

En el año de 1986 se convierte en un estándar para la ANSI y en 1987, forma parte de la ISO.

Categorías

- **DDL (Data Definition Language): Sentencias que definen y crean objetos en la BD.**
- **DML (Data Manipulation Language): Sentencias para operar sobre los datos.**
- **DCL (Data Control Language): Sentencias orientadas hacia la administración de la BD.**

DDL - Tipos de tablas

- Permanentes
- Temporales -> CREATE TEMPORARY TABLE
- Externas -> CREATE EXTERNAL TABLE

```
CREATE TABLE cliente (
id_cliente varchar(13) ,
nombre varchar(50) ,
ap_Pat varchar(50) ,
ap_Mat varchar(50),
estado varchar(25)
);
```

Tarea 11

Investigar:

Tipos de datos en postgresql:

- **numéricos**
- **caracter**
- **fecha**
- **y otros 3 que les llamen la atención**

ejemplos, capacidad, formato de representación…

DDL - Restricciones

- Check
- Not null
- Unique
- Primary keys
- Foreign keys

DDL - Restricciones

```
CREATE TABLE cliente (
id_cliente varchar(13) PRIMARY KEY,
nombre varchar(50) not null,
ap_Pat varchar(50) not null,
ap_Mat varchar(50) null,
estado varchar(25) not null
);
```

DDL - Restricciones

```
CREATE TABLE articulo (
num_Articulo int PRIMARY KEY,
nombre_Articulo varchar(30) not null,
precio numeric(4,2) CHECK (precio > 0)
);
```

Nivel columna

DDL - Restricciones

```
CREATE TABLE articulo (
num_Articulo int PRIMARY KEY,
nombre_Articulo varchar(30) not null,
precio numeric(4,2) CONSTRAINT verifica_Precio
CHECK (precio > 0) );
```

Nivel columna

DDL - Restricciones

```
CREATE TABLE articulo (
num_Articulo int,
nombre_Articulo varchar(30) not null,
precio numeric(4,2) not null,
CONSTRAINT verifica_Precio CHECK (precio > 0),
CONSTRAINT articulo_PK PRIMARY
KEY(num_articulo,nombre_Articulo));
```

Nivel tabla

DDL - Restricciones

```
CREATE TABLE articulo (
num_Articulo int,
nombre_Articulo varchar(30) not null,
precio numeric(4,2) CHECK (precio > 0),
CONSTRAINT articulo_PK PRIMARY
KEY(num_articulo, nombre_Articulo)
);
```

Mezcladas

- No action
- Restrict
- Set null
- Cascade

Aplican a borrado y actualización

```
CREATE TABLE orden (
id_Orden int not null,
fecha date not null DEFAULT now(),
id_Cliente varchar(13),
CONSTRAINT orden_PK PRIMARY KEY(id_orden),
CONSTRAINT orden_cliente_FK FOREIGN KEY (id_Cliente)
REFERENCES cliente(id_Cliente) ON DELETE CASCADE
ON UPDATE RESTRICT);
```

DDL - Valores por defecto

```
CREATE TABLE cliente (
id_cliente varchar(13) PRIMARY KEY,
nombre varchar(50) not null,
ap_Pat varchar(50) not null,
ap_Mat varchar(50) null,
estado varchar(25) not null DEFAULT ’cdmx’
);
```

Ejercicio 1_6

Generar el código SQL de las relaciones resultantes en el ejercicio 6_5

Ejercicio 1_6

Ordenes

Id_orden	Fecha	Id_cliente	Nom_cliente	Estado	Num_art	nom_art	cant	Precio
2301	23/02/11	101	Martin	Caracas	3786	Red	3	35,00
2301	23/02/11	101	Martin	Caracas	4011	Raqueta	6	65,00
2301	23/02/11	101	Martin	Caracas	9132	Paq-3	8	4,75
2302	25/02/11	107	Herman	Coro	5794	Paq-6	4	5,00
2303	27/02/11	110	Pedro	Maracay	4011	Raqueta	2	65,00
2303	27/02/11	110	Pedro	Maracay	3141	Funda	2	10,00

Ejercicio 1_6

ORDEN: (id_Orden , fecha, id_Cliente)

CLIENTE (id_Cliente, nombre_Cliente, estado)

DETALLE_ORDEN: (id_orden, no_Articulo, cantidad)

ARTICULO: (no_Articulo, nombre_Articulo, precio)

DDL - Valores secuenciales

```
CREATE SEQUENCE nombre_Sec [AS tipo_Dato]
[INCREMENT valor]
[MINVALUE valor]
[MAXVALUE valor]
[START valor]
[ [NO] CYCLE ];

SELECT nextval('nombre_Sec');
```

DDL - Valores secuenciales

```
CREATE SEQUENCE ejemplo
INCREMENT 1
MINVALUE 1
MAXVALUE 4
START 1
CYCLE;


SELECT nextval('ejemplo');
```

DDL - Índices

Un índice es una estructura de datos asociada a una tabla que permite localizar registros de manera eficiente, reduciendo el tiempo de búsqueda sobre una o más columnas.

DDL - Índices

Pueden ser clasificados:

- **por unicidad**
- **por columnas**
- **por funciones**
- **estructura interna**
- **almacenamiento**

DDL - Índices

Ventajas

- mejora del rendimiento de consultas

Desventajas

- almacenamiento
- mantenimiento
- impacto en operaciones DML

DDL - Índices

```
CREATE TABLE student
(
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    gender VARCHAR(50) NOT NULL,
    DOB datetime NOT NULL,
    total_score INT NOT NULL,
    city VARCHAR(50) NOT NULL
 )
```

DDL - Índices

```
INSERT INTO student

VALUES
(6, 'Kate', 'Female', '03-JAN-1985', 500, 'Liverpool'),
(2, 'Jon', 'Male', '02-FEB-1974', 545, 'Manchester'),
(9, 'Wise', 'Male', '11-NOV-1987', 499, 'Manchester'),
(3, 'Sara', 'Female', '07-MAR-1988', 600, 'Leeds'),
(1, 'Jolly', 'Female', '12-JUN-1989', 500, 'London'),
(4, 'Laura', 'Female', '22-DEC-1981', 400, 'Liverpool'),
(7, 'Joseph', 'Male', '09-APR-1982', 643, 'London'),
(5, 'Alan', 'Male', '29-JUL-1993', 500, 'London'),
(8, 'Mice', 'Male', '16-AUG-1974', 543, 'Liverpool'),
(10, 'Elis', 'Female', '28-OCT-1990', 400, 'Leeds');
```

DDL - Índices

```
SELECT * FROM student
```

Los registros serán recuperados en el siguiente orden:

id	name	gender	DOB	total_score	city
1	Jolly	Female	1989-06-12 00:00:00.000	500	London
2	Jon	Male	1974-02-02 00:00:00.000	545	Manchester
3	Sara	Female	1988-03-07 00:00:00.000	600	Leeds
4	Laura	Female	1981-12-22 00:00:00.000	400	Liverpool
5	Alan	Male	1993-07-29 00:00:00.000	500	London
6	Kate	Female	1985-01-03 00:00:00.000	500	Liverpool
7	Joseph	Male	1982-04-09 00:00:00.000	643	London
8	Mice	Male	1974-08-16 00:00:00.000	543	Liverpool
9	Wise	Male	1987-11-11 00:00:00.000	499	Manchester
10	Elis	Female	1990-10-28 00:00:00.000	400	Leeds

DDL - Índices

```
CREATE NONCLUSTERED INDEX IX_tblStudent_Name
ON student(name ASC)
```

DDL - Índices

```
SELECT * FROM student
```

Los registros serán recuperados en el siguiente orden:

id	name	gender	DOB	total_score	city
1	Jolly	Female	1989-06-12 00:00:00.000	500	London
2	Jon	Male	1974-02-02 00:00:00.000	545	Manchester
3	Sara	Female	1988-03-07 00:00:00.000	600	Leeds
4	Laura	Female	1981-12-22 00:00:00.000	400	Liverpool
5	Alan	Male	1993-07-29 00:00:00.000	500	London
6	Kate	Female	1985-01-03 00:00:00.000	500	Liverpool
7	Joseph	Male	1982-04-09 00:00:00.000	643	London
8	Mice	Male	1974-08-16 00:00:00.000	543	Liverpool
9	Wise	Male	1987-11-11 00:00:00.000	499	Manchester
10	Elis	Female	1990-10-28 00:00:00.000	400	Leeds

DDL - Índices

name	Row Address
Alan	Row Address
Elis	Row Address
Jolly	Row Address
Jon	Row Address
Joseph	Row Address
Kate	Row Address
Laura	Row Address
Mice	Row Address
Sara	Row Address
Wise	Row Address

DDL - Índices

CREATE [UNIQUE] INDEX nombre_Ind ON nombre_tabla [USING tipo] (columnas)

CLUSTERED nombre_Tab USING nombre_Ind;

DDL - Sinónimos

Un sinónimo es un alias de un objeto de base de datos que permite acceder a él con otro nombre, proporcionando independencia de datos y transparencia de ubicación.

CREATE SYNONYM nombre_sinonimo

FOR origen

DDL - Sinónimos

```
CREATE SYNONYM employees_view
FOR employes_view;
```

DDL - Vistas

Una vista es una tabla virtual basada en una consulta, la cual presenta datos de una o varias tablas en forma de filas y columnas sin almacenar los datos físicamente.

DDL - Vistas

Ventajas

- simplificación
- reutilización
- seguridad
- abstracción
- consistencia

Desventajas

- performance
- dependencia
- limitación en operaciones DML

DDL - Vistas

usuario	contrasenia	fecha_registro	contrasenia_previa	activo
profesor1	prof1	2022-02-01		T
profesor2	prof2	2022-02-02	1234	T
profesor3	prof3	2022-01-30	3540	F
profesor4	prof2	2022-02-02		T

DDL - Vistas

Vista donde sólo mostramos algunas columnas y las observaciones con activo = T

usuario	fecha_registro	activo
profesor1	2022-02-01	T
profesor2	2022-02-02	T
profesor4	2022-02-02	T

DDL - Vistas

CREATE [OR REPLACE] [TEMP | TEMPORARY] VIEW nombre_Vist [nombres_Columnas] AS consulta;

DDL - Mod. estructura

- Alter: Permite realizar diversas modificaciones a un objeto de la base de datos, por ejemplo, agregar/editar/eliminar una columna a una tabla, editar/agregar/eliminar constraints, modificar almacenamiento, etc.

DDL - Mod. estructura

ALTER TABLE nombre_Tabla ACCION

DDL - Eliminación objetos

- **Drop: Permite eliminar completamente un objeto de la base de datos, borrando tanto su definición como los datos asociados.**

DROP TIPO_OBJETO nombre_Objeto;

No todos los DBMS soportan operaciones DDL dentro de una transacción

Ejercicio 2_6

Agregar las restricciones necesarias a los atributos generados en el ejercicio 1_6. Al menos use 3 sentencias ALTER.
- Para la integridad referencial, considere cascade para actualización y set null para borrado

DML - Insert

La sentencia insert nos permite agregar/crear información en una tabla.

DML - Insert

```
INSERT INTO nombre_Tabla VALUES (val1, val2, …)

INSERT INTO nombre_Tabla [col1, col2, …]
VALUES (val1, val2, …)

INSERT INTO nombre_Tabla [col1, col2, …] SELECT
(col1, col2, …)
FROM nombre_Tabla
[WHERE …]
```

Consideraciones

- Considerar tipo de dato
- Cuidado con las llaves foráneas
- Restricciones not null

DML - Update

La sentencia update nos permite actualizar información de una tabla.

DML - Update

```
UPDATE nombre_tabla SET
nombre_columna = valor
[WHERE ...]
```

Consideraciones

- Considerar tipo de dato
- Cuidado con las llaves foráneas
- Puede emplearse la sentencia SELECT siempre y cuando se seleccione sólo una columna

DML - Delete

La sentencia delete nos permite borrar información de una tabla.

DML - Delete

DELETE FROM nombre_tabla [WHERE ...]

DML - Merge

La sentencia merge nos permite agregar, actualizar y borrar información de un solo movimiento.

*** A partir de la versión 15**

DML - Merge

```
MERGE INTO target_table
USING source_query
ON merge_condition
WHEN MATCH [AND condition] THEN {merge_update |
merge_delete | DO NOTHING }
WHEN NOT MATCHED [AND condition] THEN
{ merge_insert | DO NOTHING };
```

Ejercicio 3_6

- Agregar 3 registros a cada tabla creada en el ejercicio 2_6
- Actualizar el idCliente de algún registro de la tabla cliente. Explicar el resultado.
- Borrar 1 registro de la tabla donde se almacena el detalle de la orden. Explicar el resultado.

Acceso al servidor

- Pwd: numCuenta + 2 dígitos (solicitar)
- User: gpo1t_2027_1_numCuenta
- Puerto: 5432
- Host: 132.248.59.8
- BD: gpo1t_2027_1_numCuenta

Tarea 12

Investigar:

- **Definición de transacción**
- **Niveles de aislamiento en bases de datos relacionales**
- **Propiedades ACID**

Transacciones

Unidad de trabajo de una base de datos la cual agrupa una o más operaciones, garantizando la consistencia e integridad de los datos.

Control de transacciones:

- Begin: Inicia una transacción
- Commit: Confirma una transacción
- Rollback: Deshace completamente una transacción o permite regresar a un savepoint
- Savepoint: Puntos de control dentro de una transacción

Propiedades de las transacciones (ACID):

- Atomicidad: La transacción es tratada como una unidad atómica, o se ejecutan todas sus operaciones o ninguna. No hay transacciones parcialmente ejecutadas.
- Consistencia: La BD debe permanecer en un estado consistente después de una transacción. No deben presentarse efectos adversos.

Transacciones

- Aislamiento: Toda transacción debe tratarse como si fuera la única en proceso.
- Durabilidad: Los datos que son confirmados por una transacción, deben quedar almacenados sin importar las fallas que puedan presentarse en el sistema.

Niveles de aislamiento:

- Lecturas no confirmadas: Permite lecturas sucias, lo que implica que una operación realizada dentro de una transacción puede partir de cambios que aún no son confirmados por otra transacción.

Transacciones

TRANSACCION 1

SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 20

TENEMOS LECTURAS SUCIAS

SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 30

TRANSACCION 2

UPDATE ALUMNO SET EDAD = 30
WHERE NOMBRE = ‘MONSERRAT’;

— 30

ROLLBACK;

Transacciones

- **Lecturas confirmadas: Garantiza que cualquier lectura de datos está confirmada a la hora de la lectura, evita lecturas sucias. Default en postgres.**

Transacciones

TRANSACCION 1

```
SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 20
```

TENEMOS LECTURAS NO REPETIBLES

```
SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 30
```

TRANSACCION 2

```
UPDATE ALUMNO SET EDAD = 30
WHERE NOMBRE = ‘MONSERRAT’; — 30

COMMIT;
```

Transacciones

TRANSACCION 1

```
SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 20
```

```
SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 20
```

TRANSACCION 2

```
UPDATE ALUMNO SET EDAD = 30
WHERE NOMBRE = ‘MONSERRAT’; — 30
```

```
ROLLBACK;
```

Transacciones

TRANSACCION 1

SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 20

SELECT EDAD FROM ALUMNO
WHERE NOMBRE =
‘MONSERRAT’; — 20

TRANSACCION 2

UPDATE ALUMNO SET EDAD = 30
WHERE NOMBRE = ‘MONSERRAT’; — 30

Transacciones

- **Lecturas repetibles: La transacción mantiene bloqueos en todas los registros a los que hace referencia. Evita lecturas no repetibles.**

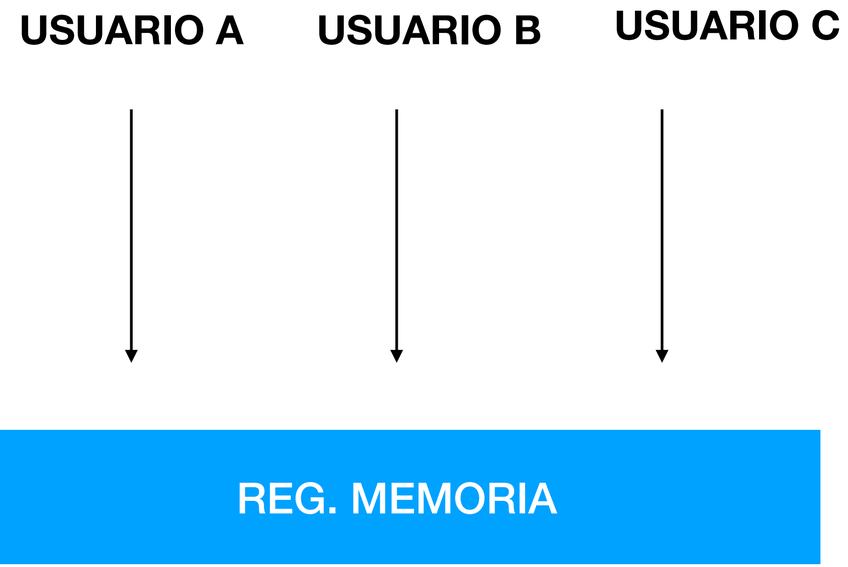
Transacciones

- **Lecturas serializables: Nivel de aislamiento más alto, en el que las transacciones dan la apariencia de ejecutarse de forma secuencial.**

Transacciones

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read uncommitted	Allowed, but not in PG	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Allowed, but not in PG	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Concurrencia



Múltiples usuarios pueden interactuar con la base de datos al mismo tiempo

Concurrencia

Capacidad de una base de datos para permitir que múltiples transacciones se ejecuten simultáneamente, garantizando la consistencia de los datos, el aislamiento entre transacciones y un uso eficiente de los recursos.

Control de concurrencia:

- Optimista: Asume que difícilmente, las transacciones se van a interferir.

“Haz ahora, valida después”

Concurrencia

- **Pesimista: Asume que los conflictos son probables, por lo que utiliza bloqueos para evitar que otras transacciones accedan a los mismos datos simultáneamente**

Concurrencia

- **MVCC: Multi Version Concurrency Control, se evitan bloqueos generando versiones de los registros cada una con algún identificador**